ReTool-MA: Multi-Agent Extension to the ReTool Reinforcement Learning Framework for Strategic Tool Use in LLMs

Ethan Babel
Johns Hopkins University
ebabel1@jh.edu

Simon Rupp
Johns Hopkins University
srupp4@jh.edu

Abstract

Large language models (LLMs) increasingly rely on external computation tools, such as code interpreters, to solve mathematically intensive reasoning tasks. Recent research has demonstrated that reinforcement learning (RL) can be used to encourage strategic tool use, vastly improving performance on complex reasoning problems. We focus on a paper called ReTool for this project which proves the effectiveness of this idea. However, ReTool employs a monolithic single-agent policy, forcing one model to learn the entire reasoning workflow (planning, code generation, and verification). Behavioral analysis of ReTool reveals that code execution failures correlate strongly with incorrect final answers, suggesting that reasoning and tool-use competence may be entangled in ways that hinder credit assignment and robustness. In this project, we introduce ReTool-MA, a multi-agent extension of the ReTool framework designed to decouple planning, execution, and verification into dedicated roles with the expectation that this will lead to improved reasoning capability.

1 Introduction

Recent work has shown that LLMs can substantially improve their mathematical reasoning capabilities by leveraging external computation tools such as Python interpreters. Instead of performing all symbolic manipulation and arithmetic via textual reasoning, a model may write code, execute it, inspect the output, and integrate the result into its reasoning process. ReTool [Feng et al., 2025] formalizes this behavior as an RL problem, training an LLM to interleave natural language reasoning with strategic code interpreter (CI) tool use. Through a combination of supervised cold-start training and PPO-based RL, ReTool achieves groundbreaking performance on AIME 2024 and 2025.

However, the original ReTool system employs a single monolithic policy π_θ, which must simultaneously handle high-level reasoning, code synthesis, and verification of tool outputs. A key limitation of this design is that code execution failures correlate strongly with incorrect final answers and that this divergence grows over the course of training.

In this work, we explore whether architectural decomposition can mitigate this issue. We introduce ReTool-MA, a multi-agent extension of the ReTool framework that separates the original policy into three distinct roles:

1. Planner - responsible for reasoning and specifying tool calls
2. Executor - responsible solely for generating and running Python code
3. Verifier - responsible for determining correctness and deciding whether revisions are needed.

We implement ReTool-MA using MARTI [Zhang et al., 2025], a flexible multi-agent workflow engine, and train the system using PPO with lightweight, role-specific reward shaping.

2 ReTool: Paper Summary

This section provides a detailed overview of the ReTool framework introduced in [Feng et al., 2025], which studies how to train LLMs with RL to strategically leverage both natural language reasoning and CI tool use. We will focus on the problem formulation, the two-stage training pipeline, and the empirical and behavioral findings that motivate our ReTool-MA extension.

2.1 ReTool Motivation

ReTool addresses the fundamental gap between purely text-based RL for reasoning and the demands of tasks that require exact computation, symbolic manipulation, and programmatic execution, such as AIME-style math problems. Let $\pi_\theta(y_{1:T} \mid x)$ denote a text-only policy over output token sequences $y_{1:T}$ conditioned on an input problem x. In conventional RL reasoning setups, the environment is purely textual, and the reward depends only on the final string, e.g.,

$$r(x, y_{1:T}) = \begin{cases} +1, & \text{if the final answer token sequence matches the ground truth,} \\ -1, & \text{otherwise.} \end{cases} \tag{1}$$

Such policies must implement all intermediate computation "in the head", which leads to error accumulation and poor sample efficiency, particularly on long-horizon problems. Empirically, text-only RL agents often overfit to textual patterns they have already seen and fail to generalize to harder math benchmarks, even when trained with strong advantages or curated reward models.

In contrast, a code interpreter can be viewed as an external tool $T : \mathcal{C} \to \mathcal{O}$ mapping code snippets $c \in \mathcal{C}$ (e.g., Python programs) to structured outputs $o \in \mathcal{O}$ (e.g., printed values, exception traces). ReTool treats access to T as an additional action modality. Rather than forcing the policy to approximate computation in text space, the agent can:

1. decide when to call T,
2. decide what code to write,
3. decide how to incorporate the resulting observation o back into its reasoning.

Previous work on tool use has relied primarily on prompting strategies or supervised fine-tuning (SFT) over static traces, which do not provide explicit credit assignment for tool-calling decisions and tend to generalize poorly out of sample. ReTool uses cold-start SFT to teach the model how to use the tool and then frames it as an RL problem in which the policy optimizes task reward. While not explicitly rewarding tool use, the policy learns to use it often in practice.

2.2 ReTool Methodology

Formally, ReTool defines an episodic Markov decision process (MDP) over trajectories that mix natural language tokens and tool calls. Let:

- x be the input problem (e.g., an AIME question),
- s_t denote the text state at step t, consisting of the conversation history,
- a_t denote the next token (or tool delimiter) sampled from the policy,
- $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$ be the full trajectory.

The policy $\pi_\theta(a_t \mid s_t)$ is a decoder-style LLM that outputs either ordinary text tokens or special `<code>` / `</code>` tags. When a closing `</code>` tag is emitted, the substring between the most recent `<code>` and `</code>` tags is extracted as a code snippet c, executed in a sandboxed Python environment, and the resulting output o (stdout, stderr, error messages) is injected into the state as

$$s_{t+1} = \text{append}(s_t, \texttt{<interpreter>}\ o\ \texttt{</interpreter>}). \tag{2}$$

From the policy's perspective, tool use is thus a stochastic sequence of discrete decisions that change the observation s_t via code execution side effects.

The training pipeline consists of two stages:

1. **Cold-start SFT.** ReTool constructs a synthetic dataset $\mathcal{D}_{\text{SFT}} = \{(x^{(i)}, y^{(i)}_{1:T_i})\}$ of code-augmented reasoning traces, where the targets include interleaved natural language reasoning, `<code>` blocks, and `<interpreter>` outputs. The model is trained to maximize the conditional log-likelihood

$$\mathcal{L}_{\text{SFT}}(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{SFT}}}\left[\sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{1:t-1})\right], \tag{3}$$

   thus bootstrapping basic competence in producing tool-augmented traces.

2. **PPO-based RL finetuning.** Starting from the SFT checkpoint, ReTool performs on-policy rollouts on math benchmarks. For each problem x, the model samples a trajectory τ until it emits a final answer in a prescribed format (e.g., `\boxed{}`). A terminal reward is assigned as

$$R(\tau) = \begin{cases} +1, & \text{if the final answer is exactly correct,} \\ -1, & \text{otherwise.} \end{cases} \tag{4}$$

   There is no intermediate shaping, so tool use only influences return through its effect on answer correctness.

The RL stage aims to maximize the traditional expected return objective

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\right]. \tag{5}$$

Let θ_{old} denote the parameters used to generate the batch of trajectories, and define the importance sampling ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{6}$$

With advantage estimates $\hat{A}_t$ computed via a value baseline or a generalized advantage estimator, the PPO objective is

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\right], \tag{7}$$

where ϵ is a clipping hyperparameter. Gradients are taken with respect to the LLM parameters, so the model is directly optimized to favor token sequences (and hence tool-use strategies) that lead to higher final reward. While effective credit assignment for tool calls is non-trivial since rewards are binary and delayed, ReTool observes strong gains over pure SFT baselines nonetheless.

2.3 ReTool Results

ReTool evaluates its approach on the AIME 2024 and 2025 benchmarks using several backbone models. After only 400 RL steps, ReTool achieves 67.0% on AIME 2024 and 49.3% on AIME 2025 with the Qwen2.5–32B-Instruct backbone. The full results are summarized below in Table 1.

Table 1: ReTool pass@1 accuracy on AIME 2024 and AIME 2025 benchmarks. Results reproduced from Feng et al. [2025].

Model	**AIME 2024 (pass@1)**	**AIME 2025 (pass@1)**
Existing Baselines		
Qwen2.5-Math-72B-Instruct	30.0	–
Qwen2.5-Math-72B-Instruct-TIR	40.0	–
Sky-T1	43.3	–
OpenAI o1-preview	44.6	37.9
DeepSeek-R1-Zero-Qwen-32B	47.0	–
QWQ-32B-Preview	50.0	33.5
s1-32B	56.7	–
CI-powered RL		
ReTool (Qwen2.5–32B-Instruct)	**67.0**	**49.3**
ReTool (DeepSeek-R1-Distill-Qwen-32B)	**72.5**	**54.3**

2.4 ReTool Output Behavioral Analysis

Beyond aggregate accuracy, ReTool conducts a detailed behavioral analysis of the learned policy. Let ρ_{code} denote the fraction of responses that contain at least one `<code>` block, and let ℓ_{resp} denote the average response length. After RL, ReTool observes:

- ρ_{code} increases to nearly 98% on evaluation sets, indicating that the policy has learned to rely on the code interpreter for most problems.
- ℓ_{resp} decreases by roughly 40% relative to the pre-RL model, reflecting a shift from verbose textual derivations to more concise explanations augmented with code.

Code snippets themselves become longer and more structured, often defining helper functions and loops to search over combinatorial possibilities or verify constraints. This suggests that the policy is not simply learning to sprinkle in trivial calls to the interpreter but is instead planning how to offload computationally demanding subproblems into code.

A key qualitative behavior is self-correction driven by interpreter feedback. When executing a snippet c yields an exception or unintended output o, the policy frequently engages in a repair loop:

1. Inspect the error message in the `<interpreter>` block.
2. Update the internal reasoning to hypothesize a fix.
3. Emit a revised code snippet c' that addresses the issue.

Empirically, many trajectories contain multiple code iterations before convergence to a successful run, which indicates that RL has taught the model to treat tool feedback as a learning signal within the episode.

However, ReTool also reveals a failure mode that is central to the motivation for our proposed multi-agent extension to this framework. When conditioning on incorrect final answers, the probability that a code snippet passes (i.e., executes without error and returns a value consistent with the model's stated intent) diverges downward over training compared to trajectories with correct answers.

3 ReTool-MA Motivation

While ReTool demonstrates that CI-powered RL can substantially improve pass@1 accuracy on AIME-style benchmarks, its behavioral analysis reveals a key limitation, namely that code execution failures are strongly coupled with incorrect final answers. As shown in Figure 1, the conditional probability that a code snippet passes given an incorrect final answer, $\mathbb{P}(\text{code pass} \mid \text{answer incorrect})$, diverges downward over training relative to $\mathbb{P}(\text{code pass} \mid \text{answer correct})$. This suggests a tight feedback loop between the model's high-level reasoning and its tool-use behavior.

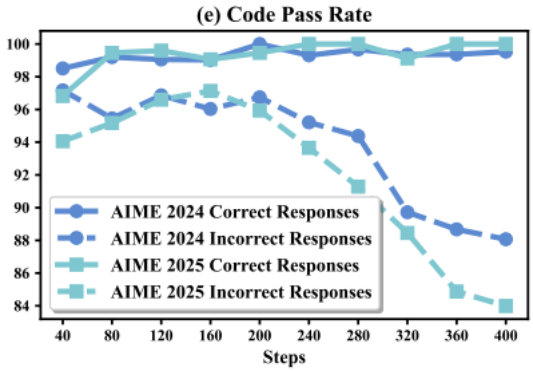

Figure 1: ReTool code pass rates conditioned on final answer correctness over RL training. Reproduced from Feng et al. [2025].

One interpretation is that code failures are a downstream effect of degraded reasoning. As problems become more complex, the single ReTool policy π_θ must simultaneously perform several cognitively demanding tasks:

1. maintain long-horizon mathematical reasoning,
2. decide when and how to call the code interpreter,
3. write syntactically valid and semantically aligned Python code,
4. verify whether tool outputs are consistent with the overall plan.

When the model's latent reasoning state drifts away from a correct solution, the same underlying representation is used to generate code. In RL terms, the policy state s_t is overloaded by its responsibilities to encode both the problem-solving context and the code synthesis context. Errors in the reasoning component of s_t might therefore induce errors in code-generation actions a_t, causing execution to fail and compounding the trajectory's deviation from high reward. In this view, degraded reasoning is the root cause of both incorrect answers and code execution failures.

A more concerning and perhaps more likely interpretation is that causality runs in the opposite direction, and incorrect code is itself a primary cause of incorrect answers. In the ReTool MDP, tool calls are high-leverage actions. A single incorrect code snippet can drastically alter trajectory τ, particularly a snippet which executes without error but produces an incorrect or unintended numeric result that the policy then incorporates into subsequent reasoning. Because rewards are sparse and only provided at the end of the episode, the RL update does not explicitly disentangle whether a low return $R(\tau)$ arose from flawed symbolic reasoning, incorrect code, a verification failure, or some combination of all three. The monolithic policy must implicitly solve this credit assignment problem across different sub-tasks, which is difficult in long-horizon RL.

Formally, we can view the ReTool policy as implementing three sub-policies

$$\pi_\theta(a_t \mid s_t) \approx \pi_\theta^{\text{plan}}(a_t^{\text{plan}} \mid s_t)\ \pi_\theta^{\text{code}}(a_t^{\text{code}} \mid s_t)\ \pi_\theta^{\text{ver}}(a_t^{\text{ver}} \mid s_t), \tag{8}$$

corresponding respectively to planning, code synthesis, and verification actions. In the original ReTool setup, these components are implicitly entangled within a single parameter vector θ and share a common hidden state. As a result, gradient updates driven by terminal rewards $R(\tau)$ simultaneously influence all three behaviors.

Regardless of the true causal structure (whether incorrect reasoning leads to code failures, or incorrect code leads to incorrect reasoning) it is clear that improving the code pass rate should improve downstream accuracy on math benchmarks. If we denote the final answer as a function of both the reasoning trajectory and tool outputs, $\hat{y} = f(\tau_{\text{text}}, \tau_{\text{tool}})$, then in the idealized limit where all code executes correctly and faithfully implements the intended computations, the remaining error can be attributed purely to high-level reasoning. In other words, raising $\mathbb{P}(\text{code pass} \mid x)$ for all tasks x contracts the space of possible failure modes and simplifies the RL objective.

These observations motivate ReTool-MA, a multi-agent extension that explicitly decomposes the monolithic policy into separate roles:

- *Planner* responsible for high-level reasoning and specifying tool calls,
- *Executor* responsible solely for generating and running code,
- *Verifier* responsible for checking consistency between the problem, plan, and tool outputs.

Conceptually, ReTool-MA can be viewed as introducing a form of factored policy:

$$\pi_\Theta(\tau) = \pi_{\theta_{\text{plan}}}^{\text{plan}}\ \pi_{\theta_{\text{exec}}}^{\text{exec}}\ \pi_{\theta_{\text{ver}}}^{\text{ver}}, \tag{9}$$

with distinct parameter sets $\Theta = \{\theta_{\text{plan}}, \theta_{\text{exec}}, \theta_{\text{ver}}\}$ and role-specific feedback. By separating planning, execution, and verification into different agents, we aim to:

- reduce the cognitive load and effective action dimensionality for each policy,
- improve credit assignment by associating tool-execution failures primarily with the Executor (and its inputs), and
- stabilize code pass rates even on trajectories where the Planner's reasoning is imperfect.

Our central hypothesis is that this architectural decomposition will increase overall code pass rates, particularly on difficult problems, and translate these gains into higher pass@1 accuracy.

4 ReTool-MA Methodology

We implement the dedicated Planner, Executor, and Verifier roles using MARTI [Zhang et al., 2025], a general-purpose multi-agent workflow engine. Due to compute budgets, each agent is backed by a Qwen2.5-3B-Instruct [Team, 2024, Yang et al., 2024] reasoning model. The workflow orchestrates their interaction through a directed graph, tool calls, and role-specific rewards.

4.1 Role Decomposition and MARTI Integration

We instantiate a custom workflow `planner` $\rightarrow$ `executor` $\rightarrow$ `verifier` inside MARTI's asynchronous DAG runtime. The Planner receives the raw math problem and must emit a structured JSON plan containing free-form reasoning, a high-level code sketch, and the anticipated numerical quantity. The Executor is prompted with the Planner JSON and is only allowed to emit executable Python. Finally, the Verifier consumes the original problem, Planner justification, and the Executor's code transcript to decide whether the trajectory should terminate or request revisions. The workflow returns a full trajectory trace, including intermediate prompts, code snippets, tool metadata, and a compact metrics object.

4.2 Local Code Execution and Instrumentation

To avoid dependence on remote execution services, we implemented a lightweight `local_python` tool that drops code into a temporary directory, runs the repository Python interpreter with strict timeouts, captures stdout/stderr, and cleans up artifacts. The tool is registered via MARTI's `ToolManager`, so both inference and RL use the exact same sandbox. Each tool call contributes to a global metrics collector.

4.3 Reward Shaping for Role-Specific Behaviors

The base ReTool reward is a sparse terminal correctness signal provided by the math grader, where each episode receives $r_{\text{env}} \in \{+1, -1\}$ depending on whether the final answer is correct. To provide more informative gradients and to force true role specialization, we introduce small additional reward terms that are applied locally at each tool invocation.

Let t index tool invocations within a trajectory and denote $\mathbb{1}[\cdot]$ for the indicator function. For the Planner, Executor, and Verifier we define rewards

$$r_t^{\text{plan}} = \alpha_{\text{succ}}\, \mathbb{1}\{\text{tool\_succ}_t\} + \alpha_{\text{spec}}\, \mathbb{1}\{\text{spec\_match}_t\} - \alpha_{\text{fail}}\, \mathbb{1}\{\text{tool\_fail}_t\}, \tag{10}$$

$$r_t^{\text{exec}} = \beta_{\text{succ}}\, \mathbb{1}\{\text{tool\_succ}_t\} + \beta_{\text{align}}\, \mathcal{O}\big(c_t^{\text{plan}}, c_t^{\text{exec}}\big) - \beta_{\text{fail}}\, \mathbb{1}\{\text{tool\_fail}_t\}, \tag{11}$$

$$r_t^{\text{ver}} = \gamma_{\text{final}}\, r_{\text{env}}\, \mathbb{1}\{t = T\} - \gamma_{\text{miss}}\, \mathbb{1}\{\text{accepted\_incorrect}_t\} - \gamma_{\text{reject}}\, \mathbb{1}\{\text{rejected\_correct}_t\}, \tag{12}$$

where c_t^{plan} is the Planner's structured code sketch at step t, c_t^{exec} is the Executor's concrete Python snippet, and T denotes the index of the final Verifier decision. The overlap term $\mathcal{O}(c_t^{\text{plan}}, c_t^{\text{exec}}) \in [0, 1]$ measures the lexical agreement between the Planner's specification and the Executor's code (normalized token-level overlap). This encourages the Executor to follow Planner instructions rather than independently solving the problem.

For each role $k \in \{\text{plan}, \text{exec}, \text{ver}\}$, the total return is

$$R^k = r_{\text{env}} + \sum_t r_t^k, \tag{13}$$

with all coefficients $\alpha., \beta., \gamma.$ chosen such that $|r_t^k| \ll |r_{\text{env}}|$. In practice, we clip the shaped rewards to the interval $[-1, 1]$ so that they act as a gentle preference over trajectories that already achieve high terminal reward, rather than overwhelming the objective.

Intuitively, these terms encourage the Planner to emit executable tool specifications, the Executor to produce code that both executes successfully and tracks those specifications, and the Verifier to catch inconsistencies between tool outputs and the global reasoning state.

Due to compute constraints, we were unable to run an exhaustive hyperparameter sweep over the role-specific reward coefficients. Instead, we expirimented with a small number of plausible settings

on preliminary runs over both AMC 12 2024 and 2025 datasets, and selected the configuration that yielded the highest code pass rates on held-out validation problems. The final coefficients used in our experiments are:

- Planner: $\alpha_{\text{succ}} = 0.2$, $\alpha_{\text{spec}} = 0.1$, $\alpha_{\text{fail}} = 0.2$,
- Executor: $\beta_{\text{succ}} = 0.3$, $\beta_{\text{align}} = 0.15$, $\beta_{\text{fail}} = 0.3$,
- Verifier: $\gamma_{\text{final}} = 1.0$, $\gamma_{\text{miss}} = 0.2$, $\gamma_{\text{reject}} = 0.2$.

5 Results

We evaluate ReTool-MA on a held-out set of AIME problems and compare its performance to the original ReTool framework. Table 2 summarizes the observed pass@1 accuracy and code pass rate for both systems.

Table 2: Comparison between ReTool and ReTool-MA.

	ReTool	**ReTool-MA**
pass@1	0.581	0.160
code pass rate	0.930	0.860
model	Qwen2.5–32B	Qwen2.5–3B

Direct comparison between ReTool-MA and the original ReTool is not strictly fair, as the two systems use models with vastly different capacities. ReTool is built on a 32B-parameter backbone, whereas ReTool-MA is implemented using a 3B model. This gap in scale has a substantial effect on reasoning ability, especially as AIME problems are known to be extremely sensitive to reasoning strength, long-horizon planning, and multi-step derivations. In such settings, model size correlates strongly with pass@1 accuracy, and large models outperform small ones by wide margins.

Given this, the low pass@1 accuracy of ReTool-MA is expected and should not be interpreted as evidence against the multi-agent architecture. Instead, the key result lies in the model's tool-use robustness. Despite being orders of magnitude smaller, ReTool-MA achieves a notably high code execution success rate of 0.86, only moderately below the 0.93 achieved by the much larger ReTool model. This suggests that the multi-agent decompisition, in particular the dedicated Executor role, helps maintain code reliability even when the underlying reasoning model is substantially weaker.

6 Discussion

The evaluation presented in this work highlights both the promise and the limitations of the ReTool-MA framework. Despite weaker reasoning capacity, the 3B ReTool-MA model is still able to maintain a relatively high code execution success rate, supporting the central motivation behind our approach, that decomposing monolithic RL into a multi-agent system has the potential to improve upon tool-use gains observed in ReTool.

Our work leads to several key takeaways:

- We successfully created a functional multi-agent adaptation of the ReTool framework using MARTI, demonstrating that role decomposition can be accomplished in practice.
- We observed that code executability remains high under multi-agent decomposition, even for a small 3B backbone model. This supports our goal of separating executable correctness from reasoning correctness.

Several promising future directions emerge from this work. A natural next step is to run controlled experiments where ReTool and ReTool-MA are evaluated using backbone models of the same size. This would isolate the effect of the multi-agent architecture from the confounding effect of model scale. More broadly, the current implementation uses manually designed shaping terms for Planner, Executor, and Verifier roles. Integrating an LLM-as-Judge module (supported natively in MARTI) would allow more flexible, semantic, and context-aware reward generation, reducing the need for hand-crafted signals.

References

Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms, 2025. URL https://arxiv.org/abs/2504.11536.

Kaiyan Zhang, Runze Liu, Xuekai Zhu, Kai Tian, Sihang Zeng, Guoli Jia, Yuchen Fan, Xingtai Lv, Yuxin Zuo, Che Jiang, Ziyang Liu, Jianyu Wang, Yuru Wang, Ruotong Zhao, Ermo Hua, Yibo Wang, Shijie Wang, Junqi Gao, Xinwei Long, Youbang Sun, Zhiyuan Ma, Ganqu Cui, Lei Bai, Ning Ding, Biqing Qi, and Bowen Zhou. Marti: A framework for multi-agent llm systems reinforced training and inference, 2025. URL https://github.com/TsinghuaC3I/MARTI.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.